

Sistemas Distribuidos y Paralelos

Trabajo Entregable – Grupo 9

Graziani Martín 02117/8

Reyes Manuel 01852/0

1. Descripción del algoritmo secuencial.....	3
2. Estrategia y descripción de las etapas de diseño paralelo.....	3
• Descomposición o Particionado:.....	4
• Comunicación:.....	4
• Aglomeración:.....	4
• Mapeo:.....	5
3. Tiempos de ejecución, métricas y análisis de escalabilidad.....	5
• Tiempos de ejecución.....	5
• Speedup.....	7
• Eficiencia.....	8
• Balance de carga.....	9
• Análisis de escalabilidad.....	10
4. Uso de IA.....	10
Herramienta: ChatGPT.....	10
Herramienta: Gemini.....	11

1. Descripción del algoritmo secuencial

El programa secuencial resuelve el problema de las N-Reinas, que consiste en ubicar N piezas reina de ajedrez en un tablero de $N \times N$ de forma que ninguna pueda atacar a las otras, es decir, dos o más reinas no pueden compartir fila, columna o diagonal en el tablero.

El algoritmo secuencial implementa una búsqueda recursiva y aprovecha las simetrías del tablero para reducir el tamaño de búsqueda.

El tablero se representa mediante un array compartido, y cada elemento del arreglo representa una fila del mismo.

El valor de cada fila es un entero usado como máscara de bits, el bit encendido (valor alto) indica en qué columna está ubicada la reina. De esta forma se asegura que solamente haya una reina por fila y no es necesario almacenar el tablero completo. El uso de máscaras de bits permite realizar las validaciones de posiciones mediante operaciones binarias, considerablemente más rápidas y eficientes que recorrer estructuras de datos convencionales.

Para determinar si un tablero es válido se utiliza un backtracking recursivo. La idea es colocar una reina en una fila y verificar qué posiciones son válidas para la siguiente fila, para luego verificar de manera recursiva. Si en alguna fila no hay una posición válida, se retrocede (backtrack).

Para verificar las posiciones válidas se usa la variable bitmap, que almacena las posiciones válidas de la fila actual. Esto se hace al negar la unión entre left (diagonales a izquierda ocupadas), right (diagonales a derecha ocupadas) y down (columnas ocupadas) y luego delimitando el resultado al tamaño del tablero.

2. Estrategia y descripción de las etapas de diseño paralelo

El algoritmo secuencial de N-Reinas fue paralelizado utilizando una arquitectura híbrida basada en MPI y pthreads, con dos procesos MPI que representan a los dos nodos utilizados del cluster proporcionado por la cátedra.

La estrategia adoptada consiste en dividir el espacio de búsqueda en subproblemas independientes, distribuyendo dichos subproblemas entre procesos MPI mediante la función MPI_Scatter, la cual distribuye un conjunto de datos entre varios procesos: primero el proceso root, que contiene todos los datos, los particiona divide entre los dos procesos (con el root también recibiendo trabajo). Luego, se utiliza pthreads dentro de cada proceso para paralelizar localmente la exploración recursiva mediante backtracking.

Según Foster y Grama, el diseño de la paralelización puede describirse siguiendo las siguientes etapas:

1. Descomposición o Particionado
2. Comunicación

3. Aglomeración

4. Mapeo

- **Descomposición o Particionado:**

La etapa de particionado consiste en descomponer el problema original en tareas independientes.

En el problema de N-Reinas, se debe decidir dónde colocar la primera reina en el tablero, para luego evaluar dónde colocar las demás reinas. Esto genera un subárbol de búsqueda, que es lo que se elige como tarea, ya que los subárboles resultan independientes entre sí.

Concretamente, una tarea consiste en:

- elegir una posición inicial válida,
- explorar posiciones para las demás reinas recursivamente,
- llevar una cuenta de las posibles soluciones.

Dentro de cada proceso, los pthreads comparten una estructura de trabajo común desde la cual obtienen las tareas pendientes. Para evitar que múltiples hilos accedan simultáneamente a dicha estructura, se utiliza un mutex para garantizar exclusión mutua durante la asignación de tareas. El mutex protege la sección crítica encargada de entregar el próximo trabajo disponible, evitando condiciones de carrera y asegurando que una misma tarea no sea asignada más de una vez.

Este paralelismo es entonces un paralelismo de datos, ya que cada tarea realiza el mismo trabajo, es decir, ejecutan el mismo código pero sobre distintos datos. La granularidad de esta solución resulta relativamente gruesa, ya que cada unidad de procesamiento puede llegar a tener un gran subárbol de búsqueda, pero aún así, seguir subdividiendo las tareas de cada unidad de procesamiento resulta contraproducente, ya que las tareas dejarían de ser independientes unas de otras e incrementando el overhead.

- **Comunicación:**

La comunicación entre procesos se realiza mediante MPI. Inicialmente el proceso root genera una lista de trabajos y la subdivide utilizando la función MPI_Scatter, lo que genera que cada proceso reciba un subconjunto de tareas para procesar localmente. Durante la etapa de cómputo no es necesario realizar comunicaciones adicionales entre procesos, ya que cada subárbol del espacio de búsqueda puede explorarse de manera independiente. Esto reduce significativamente el overhead de comunicación y favorece la escalabilidad de la solución.

Dentro de cada proceso, los hilos se comunican entre ellos solo para repartirse el trabajo y almacenar sus resultados. Finalmente, una vez que los procesos obtuvieron la cantidad de soluciones disponibles, se combinan los resultados mediante MPI_Reduce, permitiendo calcular la cantidad total de soluciones del problema.

- **Aglomeración:**

Al definir como unidad de trabajo la exploración completa de un subárbol, cada tarea conlleva una gran cantidad de cómputo. De esta manera, no resulta necesario aplicar una aglomeración adicional de tareas, ya que la propia naturaleza del algoritmo produce tareas de granularidad gruesa.

Esta estrategia permite reducir significativamente el overhead asociado a sincronización y comunicación, debido a que cada hilo realiza una cantidad considerable de trabajo local antes de requerir nueva coordinación. Si bien el problema de N-Reinas presenta un desbalance natural en la carga de trabajo, ya que algunos subárboles pueden resultar mucho más grandes que otros, la granularidad elegida permite mantener una relación adecuada entre la cantidad de tareas y las unidades de procesamiento disponibles.

- **Mapeo:**

La estrategia de mapeo utilizada en la solución es híbrida. En una primera instancia, las tareas son distribuidas estáticamente entre los procesos MPI, asignando a cada proceso un grupo de árboles para procesar localmente.

Antes de realizar el reparto, el proceso principal determina cuánto trabajo existe dentro del rango a explorar y calcula una cantidad fija de tareas por proceso, redondeando hacia arriba para que todos reciban bloques del mismo tamaño. Luego construye una estructura auxiliar con todos los trabajos reales y, en caso de ser necesario, agrega posiciones vacías como relleno para completar los bloques. De esta manera, el espacio de búsqueda queda dividido estáticamente entre los procesos MPI, aunque algunas particiones puedan contener trabajo nulo.

Dentro de cada proceso MPI, el mapeo entre tareas y threads se hace dinámicamente en un task pool, el cual consiste en que los hilos comparten una estructura de trabajos pendientes protegida mediante un mutex, desde la cual obtienen nuevas tareas a medida que finalizan las anteriores.

De esta manera, la solución combina las ventajas de ambos enfoques: un mapeo estático entre procesos MPI para disminuir costos de comunicación entre nodos, y un mapeo dinámico local entre pthreads para mejorar el aprovechamiento de los núcleos disponibles y reducir tiempos ociosos dentro de cada proceso.

3. Tiempos de ejecución, métricas y análisis de escalabilidad

- **Tiempos de ejecución**

Se miden los tiempos totales de cómputo en las distintas configuraciones del cluster, y se toman los tiempos de cada hilo para calcular luego el speedup, la eficiencia y el balance de carga.

- Con 14 Reinas
 - Secuencial: Número de resultados: 365596 - **Tiempo Total: 0.180856 segundos**
 - N=2, T=2 (4 workers): 0.096163 segundos
 - thread más lento: 0.087130 segundos
 - promedio: 0.042011 segundos
 - N=2, T=4 (8 workers): 0.059507 segundos
 - thread más lento: 0.053614 segundos
 - promedio: 0.021006 segundos
 - N=2, T=8 (16 workers): 0.058278 segundos
 - thread más lento: 0.053638 segundos
 - promedio: 0.010532 segundos
- Con 15 Reinas
 - Secuencial: Número de resultados: 2279184 - **Tiempo Total: 1.113396 segundos**
 - N=2, T=2 (4 workers): 0.572828 segundos
 - thread más lento: 0.526283 segundos
 - promedio: 0.259729 segundos
 - N=2, T=4 (8 workers): 0.330097 segundos
 - thread más lento: 0.302196 segundos
 - promedio: 0.130042 segundos
 - N=2, T=8 (16 workers): 0.321890 segundos
 - thread más lento: 0.301861 segundos
 - promedio: 0.064969 segundos
- Con 16 Reinas
 - Secuencial: Número de resultados: 14772512 - **Tiempo Total: 7.254883 segundos**
 - N=2, T=2 (4 workers): 3.636302 segundos
 - thread más lento: 3.330941 segundos
 - promedio: 1.704076 segundos
 - N=2, T=4 (8 workers): 1.991336 segundos
 - thread más lento: 1.831410 segundos
 - promedio: 0.851471 segundos
 - N=2, T=8 (16 workers): 1.940957 segundos
 - thread más lento: 1.833125 segundos
 - promedio: 0.426160 segundos
- Con 17 Reinas
 - Secuencial: Número de resultados: 95815104 - **Tiempo Total: 50.279510 segundos**
 - N=2, T=2 (4 workers): 23.929918 segundos
 - thread más lento: 22.009914 segundos

- promedio: 11.833273 segundos
 - N=2, T=4 (8 workers): 13.030255 segundos
 - thread más lento: 12.020287 segundos
 - promedio: 5.922911 segundos
 - N=2, T=8 (16 workers): 12.641772 segundos
 - thread más lento: 12.012963 segundos
 - promedio: 2.958671 segundos
- Con 18 Reinas
 - Secuencial: Número de resultados: 666090624 - **Tiempo Total: 364.624624 segundos**
 - N=2, T=2 (4 workers): 166.178920 segundos
 - thread más lento: 153.430542 segundos
 - promedio: 86.359088 segundos
 - N=2, T=4 (8 workers): 88.990710 segundos
 - thread más lento: 82.288833 segundos
 - promedio: 43.183278 segundos
 - N=2, T=8 (16 workers): 86.190496 segundos
 - thread más lento: 82.266495 segundos
 - promedio: 21.591560 segundos

En todas las configuraciones paralelas se obtuvieron la misma cantidad de soluciones que en la versión secuencial. Para evitar repeticiones innecesarias, la cantidad de soluciones no se muestra en cada configuración.

• Speedup

$$S = \frac{T_a}{T_b}$$

- T_a : Tiempo antes de la mejora.
- T_b : Tiempo después de la mejora.

Para calcular el speedup se utiliza como tiempo antes de la mejora el tiempo del proceso secuencial (N=1, T=1), o de un solo worker, y como tiempo después de la mejora los tiempos con más de un worker, respectivamente.

- Speedups con 14 Reinas:
 - N=2, T=2 (4 workers): 1.880
 - N=2, T=4 (8 workers): 3.039
 - N=2, T=8 (16 workers): 3.103
- Speedups con 15 Reinas:
 - N=2, T=2 (4 workers): 1.944
 - N=2, T=4 (8 workers): 3.373
 - N=2, T=8 (16 workers): 3.459
- Speedups con 16 Reinas:
 - N=2, T=2 (4 workers): 1.995
 - N=2, T=4 (8 workers): 3.643

- N=2, T=8 (16 workers): 3.738
- Speedups con 17 Reinas:
 - N=2, T=2 (4 workers): 2.101
 - N=2, T=4 (8 workers): 3.859
 - N=2, T=8 (16 workers): 3.977
- Speedups con 18 Reinas:
 - N=2, T=2 (4 workers): 2.194
 - N=2, T=4 (8 workers): 4.097
 - N=2, T=8 (16 workers): 4.231

Se puede observar que al aumentar la cantidad de workers, el rendimiento mejora respecto de la versión secuencial. En todos los tamaños del problema, pasar de 4 a 8 workers produce un incremento claro del speedup. Sin embargo, al pasar de 8 workers a 16 workers, la mejora es mucho menor.

Esto muestra que agregar más unidades de procesamiento no siempre produce una mejora proporcional, ya que aparecen límites asociados al desbalance de carga, la sincronización, la comunicación y la administración del trabajo.

• Eficiencia

$$E(P) = \frac{S(P)}{P}$$

- ***S***: *Speedup* utilizando *P* unidades de procesamiento.
- ***P***: Unidades de procesamiento utilizadas.

Para el cálculo de la eficiencia se utilizan los speedups escritos anteriormente, calculados con respecto al tiempo secuencial y se los divide por el número de workers o unidades de procesamiento.

- Eficiencia con 14 Reinas:
 - N=2, T=2 (4 workers): 0.470
 - N=2, T=4 (8 workers): 0.380
 - N=2, T=8 (16 workers): 0.194
- Eficiencia con 15 Reinas:
 - N=2, T=2 (4 workers): 0.486
 - N=2, T=4 (8 workers): 0.422
 - N=2, T=8 (16 workers): 0.216
- Eficiencia con 16 Reinas:
 - N=2, T=2 (4 workers): 0.499
 - N=2, T=4 (8 workers): 0.455
 - N=2, T=8 (16 workers): 0.234
- Eficiencia con 17 Reinas:

- N=2, T=2 (4 workers): 0.525
- N=2, T=4 (8 workers): 0.482
- N=2, T=8 (16 workers): 0.249
- Eficiencia con 18 Reinas:
 - N=2, T=2 (4 workers): 0.549
 - N=2, T=4 (8 workers): 0.512
 - N=2, T=8 (16 workers): 0.264

A partir de los resultados obtenidos, se observa que la eficiencia disminuye a medida que aumenta la cantidad de unidades de trabajo. Esto se debe a que el speedup logrado no crece proporcionalmente a la cantidad de hilos utilizados.

Se observa también que, para un mismo número de workers, la eficiencia mejora al aumentar el tamaño del tablero. Esto se puede explicar porque a cuánto más grande el tablero, más trabajo hay, por ende el tiempo dedicado a cómputo es mayor, y los costos de paralelización y sincronización son relativamente menores.

Por último, se observa que la eficiencia se mantiene más o menos igual con 4 y 8 unidades de procesamiento, pero disminuye considerablemente al utilizar 16. Entonces, aumentar la cantidad de hilos no resulta tan eficiente, aunque el tiempo se reduzca.

● Balance de carga

$$B = \frac{\textit{Promedio}(T)}{\textit{Máximo}(T)}$$

- **Promedio(T):** Promedio de los tiempos de ejecución de los hilos o procesos.
- **Máximo(T):** Tiempo de ejecución del hilo o proceso que más tardó.

Para calcular los balances de carga, se tomaron los tiempos de cada hilo, de éstos se elige el de mayor duración, y también se calcula el tiempo promedio de ejecución de cada configuración.

- Balance de carga con 14 Reinas:
 - N=2, T=2 (4 workers): 0.482
 - N=2, T=4 (8 workers): 0.392
 - N=2, T=8 (16 workers): 0.196
- Balance de carga con 15 Reinas:
 - N=2, T=2 (4 workers): 0.494
 - N=2, T=4 (8 workers): 0.430
 - N=2, T=8 (16 workers): 0.215
- Balance de carga con 16 Reinas:

- N=2, T=2 (4 workers): 0.512
- N=2, T=4 (8 workers): 0.465
- N=2, T=8 (16 workers): 0.232
- Balance de carga con 17 Reinas:
 - N=2, T=2 (4 workers): 0.538
 - N=2, T=4 (8 workers): 0.493
 - N=2, T=8 (16 workers): 0.246
- Balance de carga con 18 Reinas:
 - N=2, T=2 (4 workers): 0.563
 - N=2, T=4 (8 workers): 0.525
 - N=2, T=8 (16 workers): 0.262

El balance de carga es mejor en medida que se acerca a 1, lo que significaría que el tiempo promedio es igual al tiempo del hilo que más tardó, indicando una distribución de trabajo más uniforme.

Con los resultados obtenidos se nota que al aumentar el número de unidades de procesamiento, el balance de carga empeora. Esto puede explicarse por la naturaleza del problema de N-Reinas, donde cada hilo explora ramas distintas e independientes entre sí. Como estas ramas no necesariamente tienen la misma cantidad de trabajo, algunos hilos pueden finalizar antes que otros.

Por lo tanto, al incrementar la cantidad de unidades de procesamiento, es más probable que el trabajo se distribuya de manera desigual, concentrándose el cómputo en pocos hilos, aumentando la diferencia entre el tiempo del hilo más lento y el promedio.

● **Análisis de escalabilidad**

Un sistema paralelo es escalable si la combinación de la arquitectura y los algoritmos que ejecuta permite mantener la eficiencia aproximadamente constante al incrementar simultáneamente el número de unidades de procesamiento y el tamaño del problema.

Guiándose por la definición, el programa paralelo híbrido no es completamente escalable, ya que al incrementar simultáneamente el tamaño del problema (número de reinas) y la cantidad de unidades de procesamiento, la eficiencia no se mantiene constante, por más que para un mismo número de hilos la eficiencia cae, especialmente entre 8 y 16 hilos totales.

Por lo tanto, el comportamiento observado indica una escalabilidad limitada. El algoritmo paralelo aprovecha mejor los recursos cuando el tamaño del problema crece, pero el desbalance de carga y la distribución del trabajo impiden que la eficiencia se mantenga estable al aumentar la cantidad de unidades de procesamiento. Además, el problema presenta un crecimiento muy elevado al incrementarse el tamaño del tablero, por lo que los tiempos de ejecución aumentan notablemente, incluso utilizando los 16 cores disponibles.

En consecuencia, aunque la paralelización permite reducir los tiempos respecto de la versión secuencial, el aumento de recursos no logra compensar completamente el crecimiento del problema. Esto se evidencia especialmente al pasar de 8 a 16 workers,

donde la mejora en tiempo y speedup es reducida, mientras que la eficiencia y el balance de carga disminuyen considerablemente. Por este motivo, para los casos medidos, la configuración con 8 workers resulta más conveniente que la de 16 workers, ya que los tiempos de ejecución no difieren notablemente aunque se duplique el número de unidades de procesamiento.

4. Uso de IA

Herramienta: ChatGPT

Prompt	Éxito	Observación
Cómo funcionan los backtracking en N-Reinas?	Parcial	Simplifica bastante el problema, pero lo explica bien.
Cómo funcionan las simetrías?	Sí	Explica todas las rotaciones y reflexiones posibles.
Generar estructura de pthreads + MPI	Sí	Da un esqueleto básico de un programa híbrido
Scripts de bash para el cluster y recaudos que se debe tener al usarlo	Sí	Da una estructura básica de script SLURM y explica bien los recaudos para no saturar el cluster y cómo ver los resultados correctamente
Diferencias MPI_Scatterv y MPI_Scatter y cuando usar cada uno	Sí	Explica perfectamente cómo usar, sus parámetros y en qué contexto es mejor cada uno, las limitaciones del MPI_Scatter y la complejidad extra de MPI_Scatterv, y brinda ejemplos para entender mejor las diferencias
Estrategias para el análisis de Foster para un problema híbrido MPI+Pthreads	Parcial	Simplifica bastante el análisis pero da ejemplos interesantes, formas de comunicación, tipos de granularidad y formas de mapeo
Diferencia entre resultados al ejecutar en cluster o local	Sí	Tras enviarle las prestaciones del cluster, explica las diferencias con la computadora local, justificando así los tiempos de ejecución.
Problema de tareas sin asignar al usar MPI_Scatter	Parcial	Explica que el número de tareas debe ser divisible por la cantidad de procesos y si esto no pasa, se debe usar MPI_Scatterv o asignar tareas manualmente. No dice nada de hacer padding.
Desbalance de cargas con distintas configuraciones	Sí	Justifica que en N-Reinas, no todas las ramas del árbol de búsqueda cuestan lo mismo. Algunas posiciones iniciales generan muchísimas combinaciones posibles y tardan mucho más; otras se podan rápido por simetrías o conflictos.

Herramienta: Gemini

Prompt	Éxito	Observación
Cómo funcionan los backtracking en N-Reinas?	Sí	Explica el algoritmo fila a fila en el tablero de manera muy clara

Cómo funcionan las simetrías?	Sí	Explica bien las rotaciones y reflexiones y agrega algo de código para ejemplificar
Generar estructura de pthreads + MPI	Sí	Da una estructura base del código y explica como ejecutarlo
Scripts de bash para el cluster y recaudos que se debe tener al usarlo	Sí	Da reglas básicas de cómo hacer los scripts y algunas listas de buenas prácticas de programación
Diferencias MPI_Scatterv y MPI_Scatter y cuando usar cada uno	Sí	Compara las sintaxis de cada uno, explica los offsets necesarios para implementar MPI_Scatterv y detalla cuándo conviene usar cada uno
Estrategias para el análisis de Foster	Sí	Explica detalladamente cada etapa, qué es importante en cada una y qué recaudos tomar para no generar overhead innecesario
Diferencia entre resultados al ejecutar en cluster o local	Sí	Explica los tiempos de reloj de procesadores actual (cuando se corre local) y los del cluster para justificar los distintos tiempos
Problema de tareas sin asignar al usar MPI_Scatter	Sí	Explica cómo funciona MPI_Scatter y que no tiene la flexibilidad para saber qué hacer con las tareas sobrantes tras la división. Propone usar padding o directamente MPI_Scatterv
Desbalance de cargas con distintas configuraciones	No	Entendió mal la consulta y brindó alternativas incorrectas para cambiar el código. No responde el problema planteado